

PocketRights

Complete Project Specification

Offline Indian Legal-Information LLM • Research + Model Engineering + Edge Deployment

Core idea: Build a small, domain-specialized legal-information model that can run fully offline on budget Android hardware, answer everyday Indian legal questions in plain language, ground answers in real legal provisions, and expose how aggressive model compression affects legal reliability.

Primary research question: How does aggressive quantization degrade citation validity, citation relevance, numeric fidelity, critical omissions, hallucinated legal authority, and refusal behavior relative to surface-level fluency?

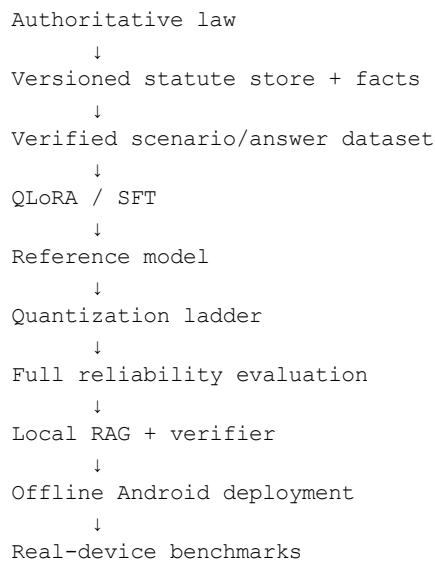
This document captures the complete project concept and development lifecycle discussed so far. It intentionally excludes the detailed technology stack and repository/folder structure so those choices can be made during implementation.

1. Executive Summary

PocketRights is an offline legal-information assistant for ordinary citizens. It is intentionally narrow: the first release focuses on consumer/e-commerce disputes, tenancy basics, traffic, salary/PF/gratuity, and insurance. It is not a general lawyer, legal representative, or universal legal database.

The system combines a versioned legal source corpus, section-level statute storage, structured numeric facts, local retrieval, a fine-tuned roughly 4B-parameter instruct model, post-generation verification, and progressively quantized variants suitable for constrained devices.

The engineering objective is to make the complete system useful offline. The research objective is to measure the accuracy-efficiency-reliability frontier as the same specialized model moves from higher precision to increasingly aggressive quantization.



2. Product Vision

The product idea is simple: a citizen asks a natural-language question and receives a concise, understandable, source-grounded explanation of what the supported legal framework says, what general next steps may be appropriate, what caveats matter, and where to escalate when appropriate.

The 'Pocket' requirement means that useful inference should happen locally. The ideal demonstration is an inexpensive Android phone in airplane mode answering a supported question with visible Act/Section citations and no cloud API call.

- Plain-English legal information rather than legal jargon.
- Real Act/Section citations rather than unsupported references.
- Clarifying questions when jurisdiction or missing facts materially change the answer.
- Explicit caveats where laws vary by state or over time.
- Clear refusal/redirect behavior outside the supported scope.
- A one-line 'information, not legal advice' notice.

3. Why We Are Building It

A language model can produce a legally plausible answer while being wrong about a Section number, monetary threshold, deadline, fine, authority, or current legal regime. Surface fluency therefore cannot be the project's main success criterion.

PocketRights treats legal reliability as something that must be measured. The project deliberately asks a harder question than whether a small model can answer legal questions: what happens to different dimensions of legal reliability when that model is compressed enough to run on constrained hardware?

- Create a controlled, narrow legal task that can be evaluated rigorously.
- Separate learned answering behavior from the legal knowledge corpus.
- Teach citation, clarification, refusal, and uncertainty behavior.
- Compare fine-tuning against a retrieval-only baseline.
- Measure the same benchmark across many precision/quantization levels.
- Deploy the resulting system on inexpensive hardware.

4. Problem Definition and Boundaries

4.1 Supported Domains

- Consumer/e-commerce: everyday disputes involving products, refunds, deficiencies, and unfair practices.
- Tenancy basics: basic landlord/tenant questions, with explicit state-law caveats.
- Traffic: everyday traffic fines/rules under the relevant central framework and applicable notifications.
- Salary/PF/gratuity: everyday employment-payment and benefits questions, subject to the current legal regime.
- Insurance: policyholder protection and grievance/escalation questions.

4.2 Out of Scope

- Criminal matters.
- Family-law disputes.
- Property-title disputes.
- Immigration.
- Complex litigation strategy.
- Individualized representation or advice requiring a qualified lawyer.

Out-of-scope behavior is trained and evaluated. A correct refusal is preferable to a confident hallucination.

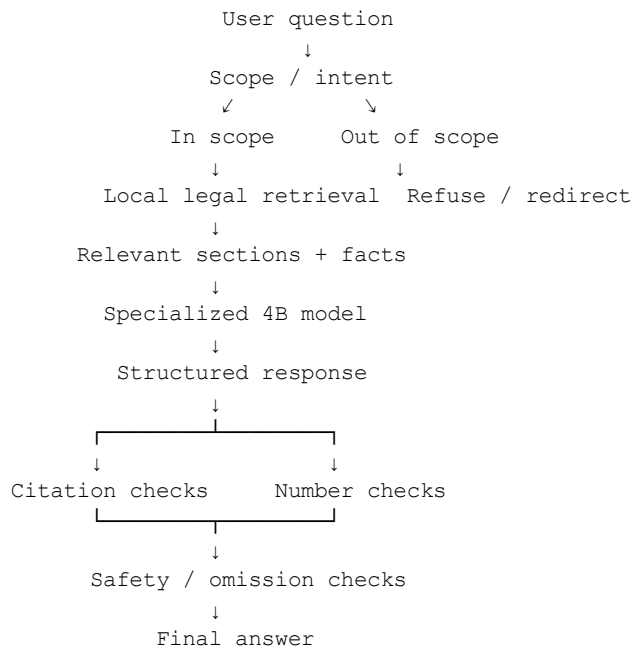
5. User Interaction and Response Contract

A normal response is conceptually structured as:

```
Answer summary
What the law says
Relevant Act + Section
Practical next steps
Caveats / missing facts
Escalation route, where applicable
Information-not-legal-advice notice
```

The answer should be useful without pretending to establish a lawyer-client relationship. It should not overstate certainty where the source corpus, jurisdiction, or facts are insufficient.

6. Overall System Concept



The model generates language; the legal corpus provides evidence; the verifier checks claims. This separation is fundamental to the design.

7. Development Lifecycle — Stage 1: Freeze the Scope

Before collecting data or training a model, define exactly what PocketRights will and will not answer.

1. Freeze the five supported verticals.
2. Define question categories within each vertical.
3. Define out-of-scope categories.
4. Define when the system must ask for state, date, or other missing information.
5. Define the response structure and disclaimer.
6. Define examples of critical omissions.

Why: a stable research task requires a stable scope. Otherwise the dataset, benchmark, and model objective continually move.

8. Stage 2: Collect Authoritative Legal Sources

Collect the authoritative legal material relevant to the five domains. Preserve raw source documents and provenance rather than only storing parsed text.

For each source, preserve source identity, retrieval date, version/snapshot information, jurisdiction, effective-date information where available, and a content hash.

The legal source must be treated as the source of truth. Secondary summaries can help discover scenarios, but they should not silently replace the authoritative corpus.

Why: every downstream artifact depends on this layer. Bad source material creates bad retrieval, bad synthetic answers, bad training data, and misleading evaluation.

9. Stage 3: Build the Statute Store

Convert the source material into section-level records so every legal provision can be independently retrieved and cited.

```
{
  act,
  section,
  title,
  text,
  jurisdiction,
  effective_from,
  effective_until,
  source,
  retrieved_at,
  version/hash
}
```

Section-level storage provides three major capabilities: retrieval, citation, and deterministic citation-existence checking.

Why: if the model produces 'Act X — Section Y', the system must be able to determine whether that reference exists without relying on another language model.

10. Stage 4: Build the Facts Table

Extract important structured facts from the legal corpus: limitation periods, monetary thresholds, fines, deadlines, percentages, deposit limits, eligibility thresholds, and other high-risk numerical facts.

```
fact_id
domain
act
section
fact_type
value
unit
jurisdiction
effective_from
source_section
verified_at
```

All values must be verified against the authoritative current corpus before being treated as ground truth. Values mentioned during planning are illustrative until verified.

Why: numbers are a major hallucination risk. A facts table lets the evaluation pipeline compare model output to a structured reference.

11. Stage 5: Build the Verification Engine

11.1 Citation existence

Check whether the cited Act and Section exist in the statute store.

11.2 Citation relevance

Check whether the cited provision actually supports the claim. This can use semantic retrieval or a judge model, but the judge must be calibrated against human labels.

11.3 Numeric verification

Extract important numerical claims and compare them against the structured facts table whenever a corresponding fact exists.

11.4 Authority verification

Validate recommended escalation authorities and contact routes against structured project information.

11.5 Scope verification

Detect cases where the candidate answer should have refused or asked for clarification.

Why: the verifier is the quality gate between synthetic generation and training. It should reject plausible-looking but unsupported answers.

12. Stage 6: Build the Scenario Matrix

Create realistic citizen scenarios rather than textbook questions. Scenarios should vary by domain, issue type, ambiguity, missing facts, difficulty, persona, and language style.

- Normal in-scope questions.
- Ambiguous questions.
- Questions missing jurisdiction or another important fact.
- Colloquial and emotionally loaded questions.
- Approximately 15% romanized-Hinglish questions.
- Out-of-scope questions.
- Clarification-required cases.
- Multi-turn cases.

Example styles include 'My seller sent the wrong phone and won't refund me', 'my landlord isn't giving my deposit back', 'company hasn't paid salary for 2 months', and romanized-Hinglish variants. The model should answer in plain English.

13. Stage 7: Generate Candidate Answers with an Open Teacher

A larger open model can generate candidate answers using the scenario plus retrieved legal provisions. The teacher should be instructed to produce a structured answer containing an answer summary, legal explanation, citations, next steps, caveats, and escalation information.

```
Scenario
+
Retrieved legal sections
```

```
↓
Open teacher
↓
Candidate structured answer
↓
Verification
```

The teacher is a generator, not a source of truth. Every candidate is subject to verification.

14. Stage 8: Verify, Repair, or Reject the Dataset

```
Generate
↓
Citation check
↓
Numeric check
↓
Claim relevance
↓
Authority check
↓
Scope/caveat check
↓
PASS → dataset
FAIL → repair/reject → verify again
```

No generated example should enter the final training dataset merely because it sounds convincing. If an answer is repaired, it must pass verification again.

Why: synthetic data scales errors. A small error rate becomes a large number of incorrect training examples when the dataset reaches tens of thousands.

15. Stage 9: Pilot Dataset and Human Audit

First build approximately 500 pilot examples spanning all five domains and major failure modes. Do not immediately generate the full 15,000–25,000 examples.

Manually audit representative examples for wrong citations, incorrect numbers, state-law assumptions, outdated regimes, missing caveats, poor language, unnecessary refusals, and overconfident claims.

Fix the generation/verification process, then regenerate affected data.

Why: the pilot validates the pipeline before scale.

16. Stage 10: Scale the Dataset

After the pilot is trustworthy, scale to the planned dataset.

```
Training target:
70% in-scope QA
15% out-of-scope refusal/redirect
10% clarification questions
5% multi-turn

Development:
~1,500
```

```
Gold test:
500
```

The exact final distribution should be audited for balance. The key principle is that all major behaviors—answering, refusing, clarifying, citing, and handling ambiguity—must be represented.

17. Stage 11: Create PocketRights-Bench

The 500-example gold set becomes PocketRights-Bench. It should be human-written, hand-verified, deduplicated against training data, and never used for training.

It should cover all domains, numerical facts, citations, state ambiguity, refusals, clarification behavior, realistic language, and critical-omission cases.

Why: the same fixed benchmark can be run against every baseline, training variant, and quantized model.

18. Stage 12: Establish Baselines

18.1 Base model, zero-shot

Evaluate the untouched base model directly on PocketRights-Bench.

18.2 Base model + local RAG

Retrieve legal provisions and provide them to the base model. This is the essential answer to 'why fine-tune instead of just using RAG?'

18.3 Fine-tuned model

Evaluate the specialized model without retrieval.

18.4 Fine-tuned model + RAG

Evaluate the hybrid system.

18.5 Frontier reference

Where practical, compare against a strong general model as a reference point, clearly distinguishing it from the offline system.

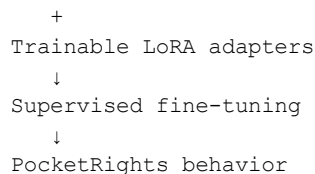
Why: without baselines, improvements cannot be attributed to fine-tuning, retrieval, or their combination.

19. Stage 13: Fine-Tune with QLoRA

The target is a roughly 4B instruct model. The exact model/version must be confirmed before implementation based on current availability, licensing, tokenizer behavior, inference support, and hardware requirements.

QLoRA loads the base model in low precision, freezes it, and trains low-rank adapter parameters. This makes specialization practical on an 8 GB-class GPU.

```
Base model
↓
Low-precision frozen weights
```



Starting configuration discussed during planning: NF4 4-bit loading, LoRA rank 16, alpha 32, sequence length around 2048, effective batch around 32 using accumulation, learning rate around $1e-4$ – $2e-4$, cosine scheduling, 2–3 epochs, and gradient checkpointing. These are starting points to validate, not fixed requirements.

20. What the Fine-Tuning Is Actually Teaching

The model should learn behavior rather than merely memorize statutes. Desired behaviors include understanding messy citizen questions, producing concise explanations, citing in a consistent format, asking for missing state information, refusing unsupported domains, communicating uncertainty, and giving appropriate next steps.

The legal corpus remains a grounding layer. Fine-tuning should not be treated as a substitute for current source retrieval.

21. Stage 14: Controlled Training Ablations

- SFT without specialized refusal examples.
- SFT with refusal/redirect examples.
- SFT with explicit structured citation behavior.
- LoRA rank comparisons such as $r=8$, $r=16$, and $r=32$.
- Additional controlled training variations where resources justify them.

The main questions are whether refusal training improves safety without causing over-refusal, whether citation-structure training improves citation reliability, and how much adaptation capacity is needed.

DPO/RLHF is intentionally excluded from PocketRights. Preference optimization is reserved for the separate RLHF project.

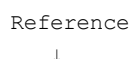
22. Stage 15: Evaluate the Fine-Tuned Models

Run every training candidate against the same held-out benchmark. Evaluation should focus on legal reliability rather than only generic language quality.

A strong candidate should improve supported-domain behavior while maintaining acceptable refusal calibration and not catastrophically destroying general capabilities.

23. Stage 16: Build the Quantization Ladder

The strongest fine-tuned model becomes the reference model. Then create increasingly compressed versions.



```

Q8_0
↓
Q6_K
↓
Q5_K_M
↓
Q4_K_M
↓
Q3_K_M
↓
IQ2-class extreme compression

Additional 4-bit comparisons:
AWQ INT4
GPTQ INT4

```

The goal is not simply to find the smallest model. It is to discover where compression begins to make the system unreliable and whether different quantization methods behave differently at similar bitwidths.

24. Stage 17: The Evaluation Suite

24.1 Citation Validity Rate

Percentage of cited Act/Section references that exist in the statute store. Primarily deterministic.

24.2 Citation Relevance

Percentage of cited provisions that genuinely support the associated claim.

24.3 Numeric Fidelity

Correctness of important numbers against the verified facts table.

24.4 Critical-Omission Rate

Rate at which answers omit information judged essential for the scenario, such as state caveats, key eligibility conditions, or required escalation routes.

24.5 Hallucinated-Authority Rate

Rate at which the model invents Acts, Sections, regulations, authorities, or helplines.

24.6 Refusal Calibration

Measure out-of-scope recall and in-scope over-refusal separately.

24.7 General Capability Retention

Use small MMLU/IFEval slices as secondary checks that specialization has not severely damaged general instruction-following ability.

25. Stage 18: Judge-Model Calibration

Some evaluation dimensions cannot be solved with deterministic lookup. Citation relevance and critical omission may require a judge model.

Use a judge from a different model family than the teacher where practical. Human-label a representative subset of the gold set and compare judge decisions with human decisions.

Report judge-human agreement. The judge should be treated as an evaluation instrument, not an unquestionable oracle.

26. Stage 19: The Signature Quantization Experiment

Run exactly the same PocketRights-Bench across every quantization rung. Keep questions, retrieval inputs, evaluation logic, and reporting methodology consistent.

```
Same 500-question benchmark
      ↓
BF16 → Q8 → Q6 → Q5 → Q4 → Q3 → IQ2
      ↓
Citation validity
Citation relevance
Numeric fidelity
Critical omissions
Hallucinated authority
Refusal calibration
Overall quality
```

This produces the project's central reliability curves. The key result is the point at which the model becomes meaningfully less reliable even if its outputs still look fluent.

27. Research Hypotheses

- H1: Aggressive quantization degrades legal reliability before obvious surface-fluency degradation.
- H2: Numeric fidelity is more sensitive to aggressive compression than broad response quality.
- H3: Citation reliability decreases at sufficiently aggressive quantization levels.
- H4: Local retrieval partially compensates for some quantization-induced factual degradation.
- H5: Fine-tuning and retrieval provide complementary benefits, with the hybrid configuration potentially strongest.

These are hypotheses to test, not conclusions to force. A result that disproves a hypothesis is valuable if the experiment is controlled and reproducible.

28. Stage 20: Local RAG

The deployed system should retrieve legal provisions locally rather than relying entirely on model memory.

```
User question
      ↓
Local retrieval
      ↓
Relevant Act / Section records
      ↓
Context assembly
      ↓
Fine-tuned model
      ↓
Structured answer
```

The retrieval corpus should be versioned separately from the model. This makes future corpus updates conceptually independent of model retraining.

29. Stage 21: Post-Generation Verification

```
Generated answer
↓
Citation validation
↓
Numeric validation
↓
Scope / safety validation
↓
PASS → display
FAIL → regenerate / clarify / refuse
```

The system should prefer uncertainty, clarification, or refusal over silently presenting an unsupported legal claim. Verification is especially important for citations and high-risk numeric statements.

30. Stage 22: Offline Android Deployment

The model is converted into a mobile-friendly local inference format and integrated into an Android application. The primary target is a budget phone with approximately 6 GB RAM. A roughly 4B Q4-class configuration is a planning target, not a guaranteed requirement; real-device testing determines feasibility.

```
Android app
↓
Local inference runtime
↓
Quantized model
+
Local legal retrieval
+
Citation / fact verification
↓
Offline answer
```

The application must remain functional with internet connectivity disabled.

31. Stage 23: Offline Demonstration

The flagship demo is intentionally simple: enable airplane mode, ask a supported question, and receive a useful answer with a real legal citation.

The demo proves that the project is not merely a cloud wrapper around a large API. It demonstrates the actual edge-inference objective.

32. Stage 24: Device Benchmarking

Benchmark the selected model family on the target Android device and useful reference hardware.

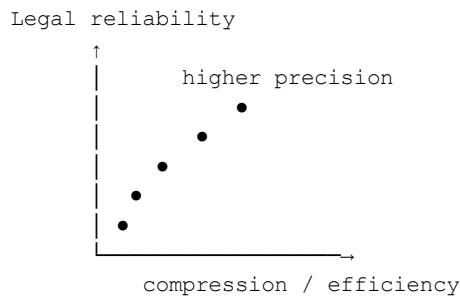
- Model size on disk.
- Peak RAM usage.
- Time to first token.
- Decode tokens per second.
- End-to-end response time.
- Sustained-generation performance.

- Thermal throttling where measurable.
- Task-level reliability on the legal benchmark.

The reference environments discussed are a budget Android phone, a Mac M4 Pro, an RTX 4060 system, and optionally a Raspberry Pi 5.

33. Choosing the Production Configuration

The production model should be selected using a multi-objective trade-off, not by model size alone.



For example, if a smaller model gains speed but loses too much numeric fidelity or citation reliability, the larger configuration may be the correct product choice. The actual decision must come from measurements.

34. Legal Risk and Versioning

34.1 State variation

Tenancy and some traffic rules are sensitive to state-specific law, notifications, and implementation. The model must ask for state information when it materially affects the answer.

34.2 Labour-law changes

The employment vertical must verify the regime in force at the corpus snapshot date. Historical Acts, newer labour codes, rules, and implementation notices must not be blended without temporal context.

34.3 Legal drift

Every model release should identify its legal corpus snapshot date and version. The system should never imply that a model is permanently current.

34.4 Numeric drift

Every critical number must be verified against the dated corpus. Planning examples are not automatically ground truth.

34.5 Scope drift

The product should not gradually become a general legal adviser through prompt interpretation alone. Expanding the scope should be a deliberate new project version with new sources and benchmarks.

35. Complete Development Lifecycle

- 1. Define and freeze legal scope
- ↓
- 2. Collect authoritative legal sources
- ↓
- 3. Preserve raw sources + provenance
- ↓
- 4. Build section-level Statute Store
- ↓
- 5. Build verified Facts Table
- ↓
- 6. Build citation/numeric/scope verifier
- ↓
- 7. Create ~500 pilot scenarios
- ↓
- 8. Generate candidate answers with open teacher
- ↓
- 9. Verify / repair / reject
- ↓
- 10. Human-audit pilot
- ↓
- 11. Scale verified training data
- ↓
- 12. Build isolated PocketRights-Bench
- ↓
- 13. Run zero-shot + RAG baselines
- ↓
- 14. QLoRA fine-tuning
- ↓
- 15. Training ablations
- ↓
- 16. Fine-tuned model evaluation
- ↓
- 17. Select reference model
- ↓
- 18. Quantization ladder
- ↓
- 19. Full benchmark at every rung
- ↓
- 20. Analyze reliability/compression curves
- ↓
- 21. Local RAG inference
- ↓
- 22. Mobile conversion
- ↓
- 23. Offline Android app
- ↓
- 24. Real-device benchmarks
- ↓
- 25. Select production configuration
- ↓
- 26. Final analysis + release

36. Why the Order Matters

Stage	Question answered
Legal corpus	What is the authoritative information?
Statute Store	How do we represent and retrieve provisions reliably?

Facts Table	How do we mechanically check critical numbers?
Verifier	How do we prevent bad synthetic examples from entering training?
Dataset	How do we teach the model desired behavior?
Baselines	Do we actually need fine-tuning?
QLoRA	Can a small model specialize effectively?
Ablations	Which training choices cause improvements?
Quantization	How far can we compress the model?
Reliability study	What breaks first under compression?
Local RAG	Can current legal sources ground answers locally?
Android	Can the complete system run offline on constrained hardware?
Device benchmark	What is the practical accuracy/latency/memory frontier?

37. Final Deliverables

37.1 Data

- Versioned legal source corpus.
- Section-level Statute Store.
- Verified structured Facts Table.
- Verified training dataset.
- Development set.
- PocketRights-Bench gold set.

37.2 Model

- Fine-tuned PocketRights model.
- Higher-precision reference version.
- Quantized GGUF ladder.
- Comparable 4-bit AWQ/GPTQ variants where feasible.

37.3 Evaluation

- Zero-shot and RAG baselines.
- Training ablations.
- Quantization degradation curves.
- Citation, numeric, omission, hallucination, and refusal analysis.
- Judge-human agreement analysis.
- Device performance benchmarks.

37.4 Product

- Offline Android application.
- Airplane-mode demonstration.
- Model card and limitation documentation.
- Legal corpus/model release version information.

38. Definition of Done

- Supported domains and boundaries are documented.

- The legal corpus has provenance and a dated snapshot.
- Sections are independently retrievable and citation-checkable.
- Critical numeric facts are structured and verified.
- Synthetic training examples pass automated verification.
- Training and gold test data are separated.
- PocketRights-Bench is human-verified.
- Zero-shot and RAG baselines are measured.
- The fine-tuned model and training procedure are documented.
- Meaningful training ablations have been performed.
- Multiple quantization levels are evaluated on the same benchmark.
- Reliability is measured separately from surface fluency.
- The selected model runs locally on the target Android device.
- Latency, memory, model size, and sustained performance are measured.
- Legal-version limitations are documented.
- The final analysis explains the accuracy-efficiency-reliability trade-off.

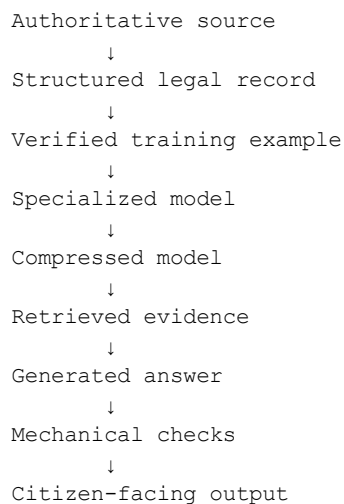
39. What Makes It a Research Project

PocketRights becomes a research project through controlled questions, explicit hypotheses, a fixed gold benchmark, baselines, ablations, independent evaluation, and systematic quantization experiments. The central contribution is not a claim that one model is universally best. It is an empirical study of which reliability properties degrade under compression and whether retrieval changes that behavior.

PocketRights-Bench can also be a contribution in its own right because it targets realistic Indian everyday-rights scenarios, including ambiguity, state dependence, numeric facts, citations, refusal behavior, and colloquial/romanized-Hinglish inputs.

40. What Makes It an Engineering Project

The engineering challenge spans the complete lifecycle: authoritative data acquisition, structured legal representation, synthetic-data quality control, supervised specialization, model compression, local retrieval, verification, mobile inference, and real-device measurement.



Every layer has a distinct failure mode. The project is therefore designed around traceability and verification rather than treating the language model as an oracle.

41. Final Project Narrative

PocketRights is an offline, domain-specialized Indian legal-information system built around a small language model and a versioned legal knowledge layer. It combines supervised specialization, local retrieval, mechanical verification, aggressive model compression, and edge deployment.

The project starts with authoritative legal sources and ends with a real Android device operating without the internet. In between, it creates a controlled benchmark and uses that benchmark to investigate how quantization changes different dimensions of legal reliability.

```
Can a small language model provide useful,  
source-grounded everyday legal information  
on inexpensive hardware?
```

```
And when we compress that model aggressively  
enough to make it practical,
```

```
which properties fail first:
```

```
fluency,  
factual accuracy,  
citations,  
numbers,  
critical safeguards,  
or refusal behavior?
```

```
PocketRights is built to measure the answer.
```

The strongest final outcome is therefore a reproducible map of the trade-off between model size, latency, general quality, and safety-relevant legal reliability, accompanied by a working offline demonstration.

42. Implementation Boundary

This document deliberately does not prescribe the detailed technology stack, package choices, cloud services, repository/folder structure, or exact implementation libraries. Those decisions are left to the engineer.

What this document freezes is the project problem, scope, data lifecycle, experimental logic, evaluation methodology, deployment objective, and definition of done. The implementation can evolve without losing the research question.

PocketRights — from authoritative law to a verified offline model.